# Lab Experiment 6: Dynamic Programming

## Part 1: Matrix Chain Multiplication

### Aim

To write a program using Dynamic Programming that finds the minimum number of scalar multiplications needed to multiply a chain of matrices, displays the optimal parenthesization, and displays the minimum cost, with the input taken from the user.

### Theory

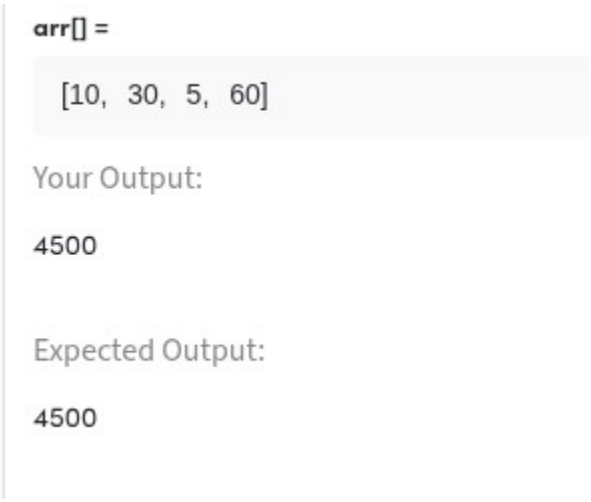
Matrix multiplication is associative, so the order of bracketing changes the total number of scalar multiplications but not the result. Matrix Chain Multiplication uses a table m[i][j] = min over k of (m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]), filled by increasing chain length, and a second table s[i][j] stores the best split point k so the optimal bracketing can be rebuilt. Time complexity is O(n^3) and space complexity is O(n^2).

### Code

```
class Solution {
  public:
    int mcm(vector<int>&arr,int i,int j,vector<vector<int>>&table){
        if(i+1==j){
            return 0;     // only one matrix
        }
        if(table[i][j]!=-1){
            return table[i][j];  // already exist in the table not to compute it again
        }
        int res=INT_MAX;
        for(int k=i+1;k<j;k++){
            int curr=mcm(arr,i,k,table)+mcm(arr,k,j,table)+arr[i]*arr[k]*arr[j];
            res=min(curr,res);
        }
        table[i][j]=res;
        return res;
    }
    int matrixMultiplication(vector<int> &arr) {
        int n=arr.size();
        vector<vector<int>>table(n,vector<int>(n,-1));
        return mcm(arr,0,n-1,table);

    }
};
```

**Output**




## Part 2: Longest Common Subsequence (LCS)

### Aim

To write a program using Dynamic Programming that finds the length of the Longest Common Subsequence of two input strings, displays the actual subsequence, and compares the Dynamic Programming approach with the brute-force recursive approach in terms of time complexity.

### Theory

A subsequence keeps the relative order of characters but need not be contiguous. The LCS table is filled with L[i][j] = L[i-1][j-1] + 1 if the characters match, otherwise max(L[i-1][j], L[i][j-1]), and the actual subsequence is recovered by backtracking from L[m][n]. The DP approach takes O(m*n) time and space, whereas the brute-force recursive approach checks all 2^m subsequences and takes O(2^(m+n)) time in the worst case because it solves the same subproblems repeatedly.

**Code**

```
class Solution {
  public:
    int lcs(string &s1, string &s2) {
       int r=s1.length();
           int c=s2.length();

           vector<vector<int>>dp(r+1,vector<int>(c+1,0));

           for(int i=0;i<r;i++){
               for(int j=0;j<c;j++){
                   if(s1[i]==s2[j]){
                       dp[i+1][j+1]=1+dp[i][j];
                   }
                   else{
                       dp[i+1][j+1]=max(dp[i][j+1],dp[i+1][j]);
                   }
               }
           }
              return dp[r][c];

    }
};
```

**Output**

s1 =

ABCDGH

s2 =

AEDFHR

Your Output:

3

Expected Output:

3

## Conclusion

Dynamic Programming reduces the exponential cost of both problems to polynomial time by storing and reusing the solutions of overlapping subproblems.